Ансамбли моделей.
Бэггинг, бустинг, etc

Majority vote

- Как выглядит логотип факультета компьютерных наук?
- Какой из двух логотипов более старый?
- Как выглядит корпус Вышки в Перми?
- Покоординатный спуск — это метод оптимизации 1-го или 2-го порядка?

Majority vote

- Дано: N базовых алгоритмов $b_1(x), \ldots, b_N(x)$
- Каждый хотя бы немного лучше случайного угадывания
- Композиция: класс, за который проголосовало больше всего базовых алгоритмов

$$a(x) = \arg\max_{y \in \mathbb{Y}} \sum_{n=1}^{N} [b_n(x) = y]$$

Усреднение наблюдений

- Сколько лет факультету компьютерных наук?
- Сколько метров в 1 сажени?
- Сколько лет лектору?
- Сколько всего стран в мире?

Усреднение наблюдений

- Наблюдение: усреднение результатов повышает их точность
- Измерение артериального давления
- Измерение скорости света
- Усреднение соседних пикселей изображения

Усреднение наблюдений

- Дано: N базовых алгоритмов $b_1(x), \ldots, b_N(x)$
- Каждый хотя бы немного лучше случайного угадывания
- Композиция:

$$a(x) = \frac{1}{N} \sum_{n=1}^{N} b_n(x)$$

Композиции алгоритмов

- Базовые алгоритмы: $b_1(x), \ldots, b_N(x)$
- Композиция: $a(x)$

- Как по одной и той же выборке обучить N различных моделей?

Рандомизация

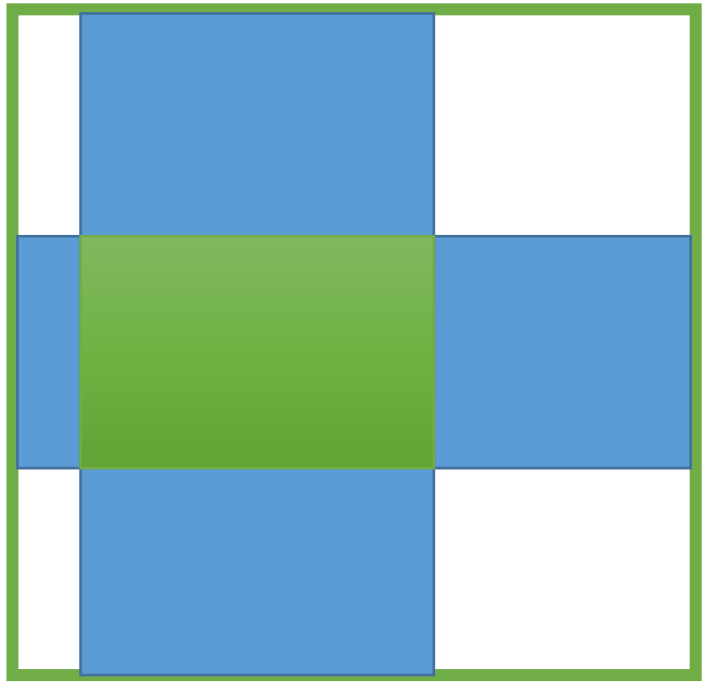
- Популярный подход: бутстрап
- Выбираем из обучающей выборки ℓ объектов с возвращением
- Пример: $\{x_1, x_2, x_3, x_4\} \to \{x_1, x_2, x_2, x_4\}$

Рандомизация

- Другой подход: выбор случайного подмножества объектов
- Гиперпараметр: размер подмножества

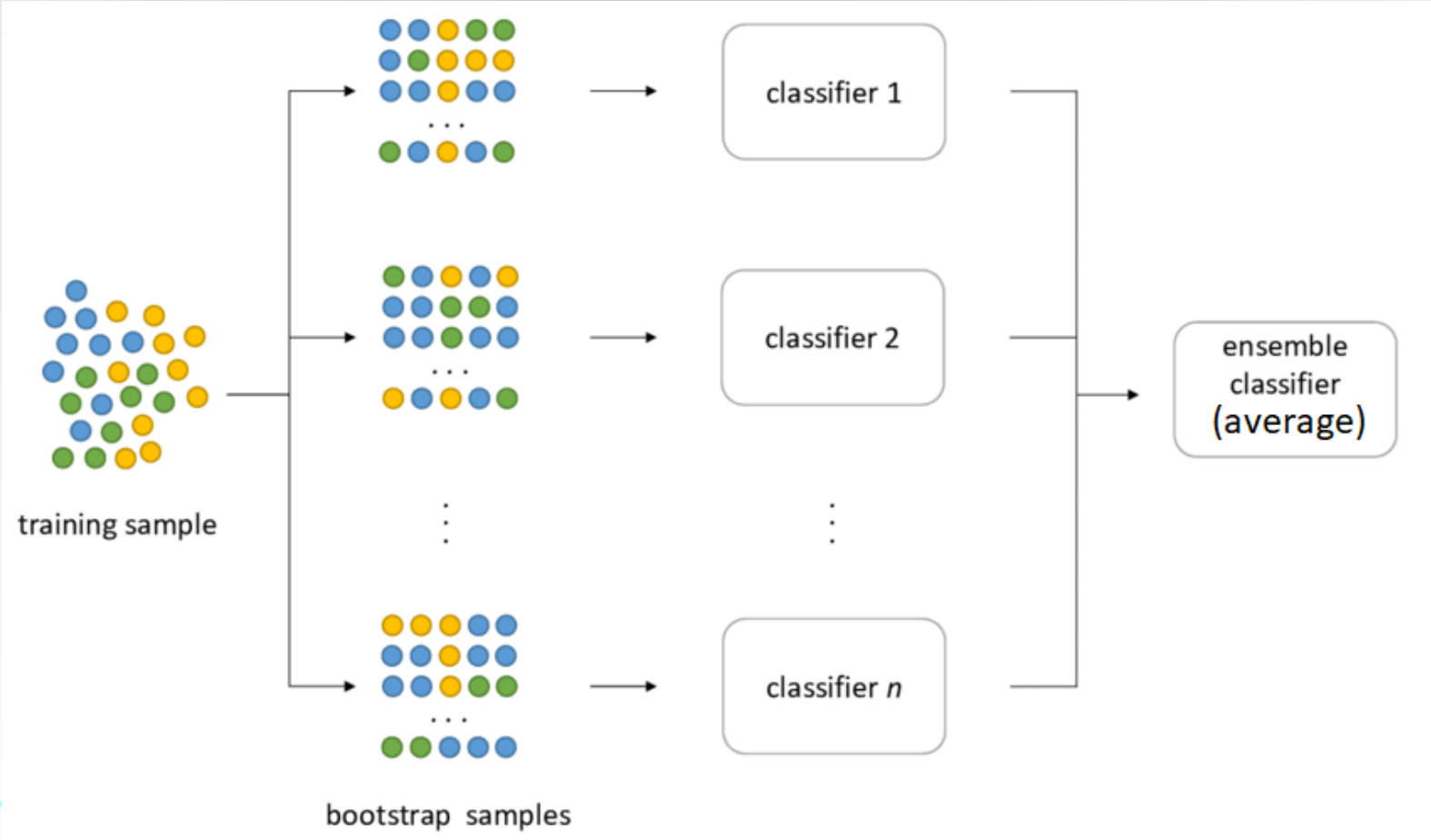
Виды рандомизации

- Бэггинг: обучаем на случайной подвыборке
- Метод случайных подпространств: обучаем на случайном подмножестве признаков
- Размер подвыборки/подмножества — гиперпараметр



БЭГГИНГ (BOOTSTRAP AGGREGATION)

$$a(x) = \frac{1}{N}\sum_{j=1}^{N} b_j(x)$$



БЭГГИНГ (BOOTSTRAP AGGREGATION)

С помощью бутстрэпа мы получили выборки $X_1, \dots, X_N$.

- Обучим по каждой из них модель – получим базовые алгоритмы $b_1(x), \dots, b_N(x)$.
- Построим новую функцию регрессии:

$$\boldsymbol{a(x) = \frac{1}{N} \sum_{j=1}^{N} b_j(x)}$$

Рандомизация

- Этого недостаточно
- Как можно рандомизировать сам процесс построения дерева?

Поиск разбиения

- Пусть в вершине m оказалась выборка X_m
- $Q(X_m, j, t)$ — критерий ошибки условия $[x^j \leq t]$
- Ищем лучшие параметры j и t перебором:

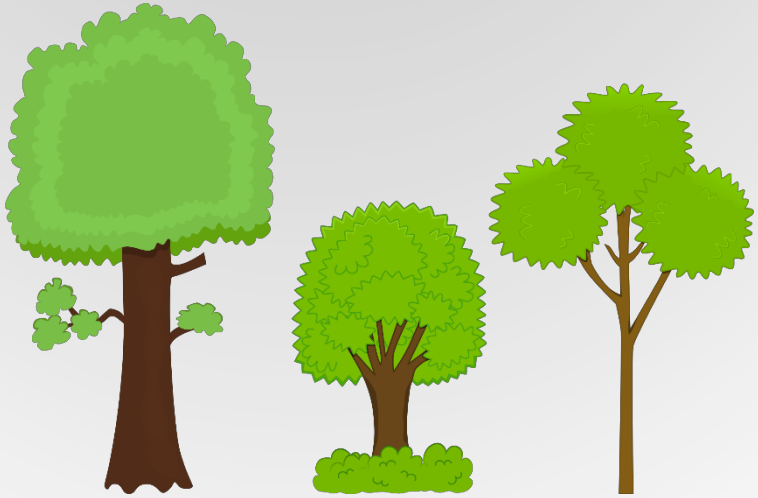
$$Q(X_m, j, t) \to \min_{j,t}$$

- Случайный лес: выбираем j из случайного подмножества признаков размера q



СЛУЧАЙНЫЙ ЛЕС (RANDOM FOREST)

- Возьмем в качестве базовых алгоритмов для бэггинга ***решающие деревья***, т.е. каждое случайное дерево $b_i(x)$ построено по своей бутстрепной подвыборке X_i.
- В каждой вершине дерева будем искать ***разбиение не по всем признакам, а по подмножеству признаков***.
- Итоговая композиция имеет вид $\boldsymbol{a(x) = \frac{1}{N}\sum_{j=1}^{N} b_j(x)}$.



RANDOM FOREST – ПРАКТИЧЕСКИЕ РЕКОМЕНДАЦИИ

- Если p – количество признаков, то при классификации обычно берут $m = [\sqrt{p}]$, а при регрессии - $m = [\frac{p}{3}]$ признаков
- При классификации обычно дерево строится, пока в листе не окажется $n_{min} = 1$ объект, а при регрессии $n_{min} = 5$

RANDOM FOREST

Алгоритм 3.1. Random Forest

1: **для** $n = 1, \ldots, N$

2: Сгенерировать выборку $\tilde{X}_n$ с помощью бутстрэпа

3: Построить решающее дерево $b_n(x)$ по выборке $\tilde{X}_n$:

- дерево строится, пока в каждом листе не окажется не более $n_{\min}$ объектов
- при каждом разбиении сначала выбирается m случайных признаков из p, и оптимальное разделение ищется только среди них

4: Вернуть композицию $a_N(x) = \frac{1}{N} \sum_{n=1}^{N} b_n(x)$

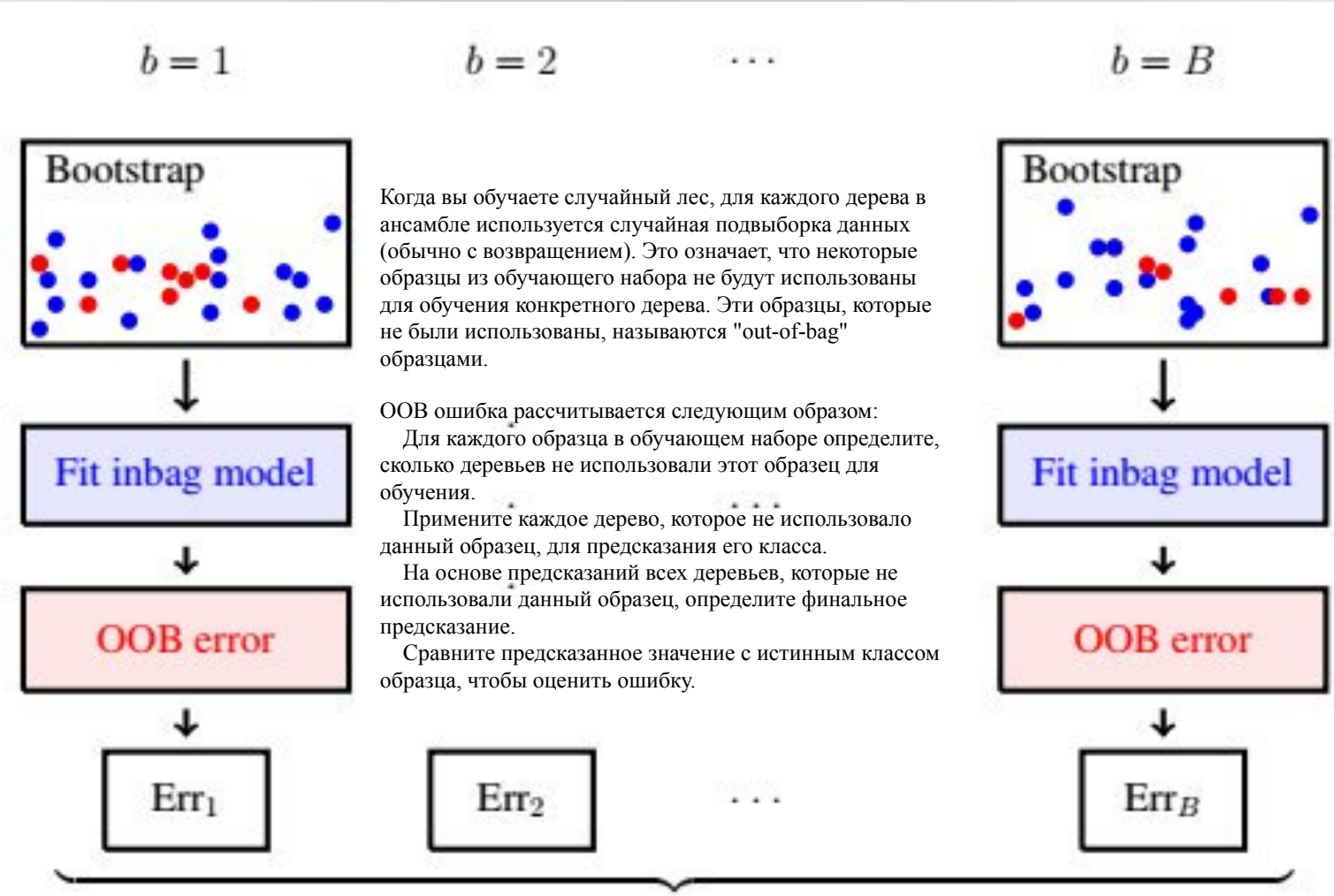
СМЕЩЕНИЕ И РАЗБРОС У БЭГГИНГА

Утверждение.

1) ***Бэггинг не ухудшает смещенность модели****, т.е. смещение* $a_N(x)$ *равно смещению одного базового алгоритма.*

2) Если базовые алгоритмы некоррелированы, то ***дисперсия бэггинга*** $a_N(x)$ ***в N раз меньше дисперсии отдельных базовых алгоритмов.***

OUT-OF-BAG ОШИБКА



Когда вы обучаете случайный лес, для каждого дерева в ансамбле используется случайная подвыборка данных (обычно с возвращением). Это означает, что некоторые образцы из обучающего набора не будут использованы для обучения конкретного дерева. Эти образцы, которые не были использованы, называются "out-of-bag" образцами.

OOB ошибка рассчитывается следующим образом:

Для каждого образца в обучающем наборе определите, сколько деревьев не использовали этот образец для обучения.

Примените каждое дерево, которое не использовало данный образец, для предсказания его класса.

На основе предсказаний всех деревьев, которые не использовали данный образец, определите финальное предсказание.

Сравните предсказанное значение с истинным классом образца, чтобы оценить ошибку.

$$\mathrm{Err}_{\mathrm{oob}} = \frac{\mathrm{Err}_1 + \cdots + \mathrm{Err}_B}{B} = \frac{1}{B}\sum_{b=1}^{B} \mathrm{Err}_b$$

OUT-OF-BAG ОШИБКА

- Каждое дерево в случайном лесе обучается по некоторому подмножеству объектов
- Значит, для каждого объекта есть деревья, которые на этом объекте не обучались.

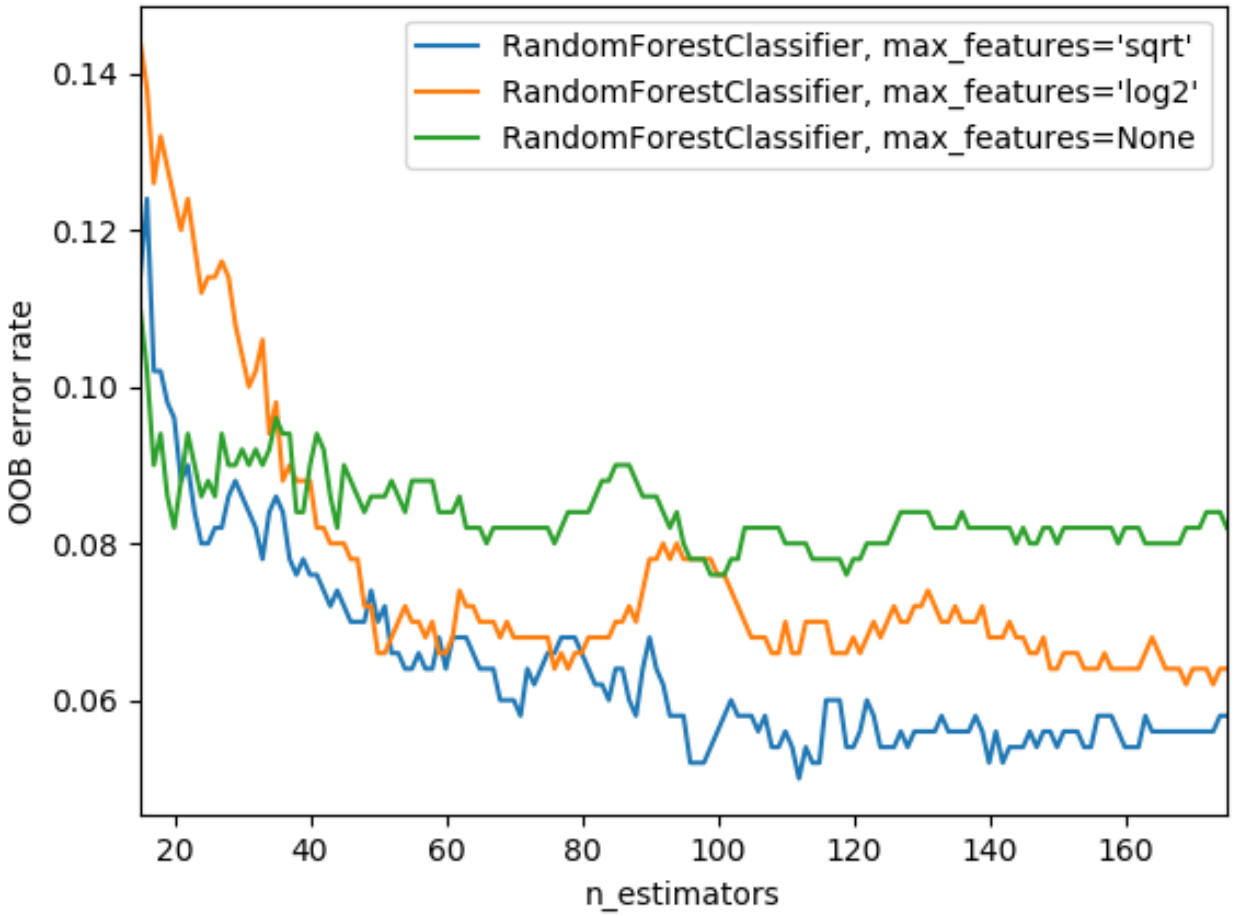
Out-of-bag ошибка:

$$OOB = \sum_{i=1}^{l} L(y_i, \frac{\sum_{n=1}^{N}[x_i \notin X_n]\, b_n(x_i)}{\sum_{n=1}^{N}[x_i \notin X_n]})$$

Утверждение. *При $N \to \infty$ OOB оценка стремится к leave-one-out оценке.*

OOB-SCORE

По графику out-of-bag ошибки можно, например, подбирать количество деревьев в случайном лесе



КАЛИБРОВКА ПЛАТТА

- Пусть есть два класса, $Y = \{+1, -1\}$

Задача: для классификатора $a(x)$, предсказывающего значения из отрезка $[0, 1]$, либо предсказывающего класс (+1 или -1), сделать калибровку, чтобы предсказания были вероятностями $p(y = +1|x)$.

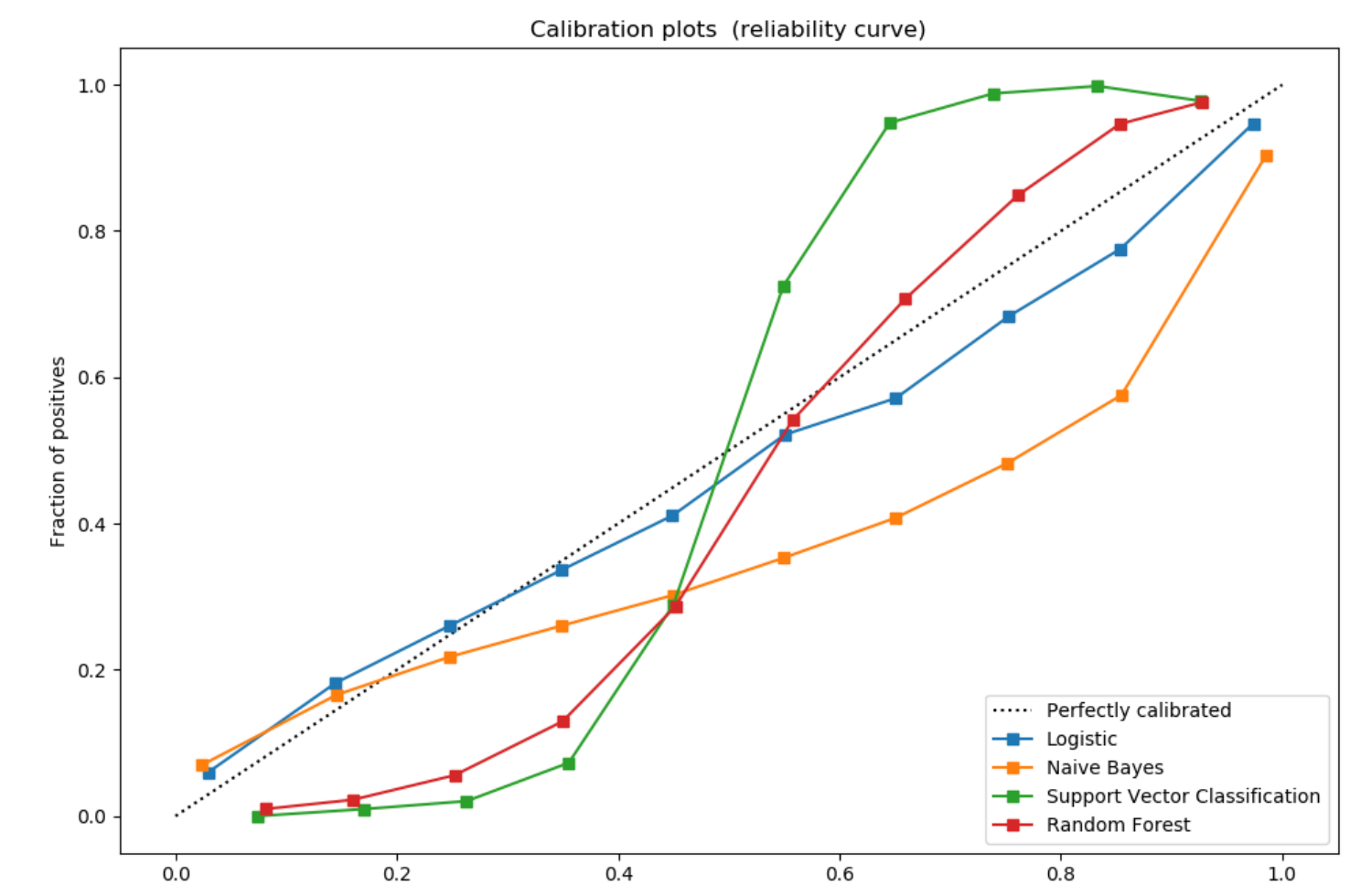
КАЛИБРОВКА ПЛАТТА

- Пусть есть два класса, $Y = \{+1, -1\}$

Задача: для классификатора $a(x)$, предсказывающего значения из отрезка $[0, 1]$, либо предсказывающего класс (+1 или -1), сделать калибровку, чтобы предсказания были вероятностями $p(y = +1|x)$.

Идея: обучаем логистическую регрессию на ответах классификатора $a(x)$.

ПРИМЕР ИЗ SKLEARN



КАЛИБРОВКА ПЛАТТА

- Пусть есть два класса, $Y = \{+1, -1\}$

Задача: для классификатора $a(x)$, предсказывающего значения из отрезка $[0, 1]$, либо предсказывающего класс (+1 или -1), сделать калибровку, чтобы предсказания были вероятностями $p(y = +1|x)$.

Идея: ***обучаем логистическую регрессию на ответах классификатора*** $a(x)$.

КАЛИБРОВКА ПЛАТТА

- Пусть есть два класса, $Y = \{+1, -1\}$

Задача: для классификатора $a(x)$, предсказывающего значения из отрезка $[0, 1]$, либо предсказывающего класс (+1 или -1), сделать калибровку, чтобы предсказания были вероятностями $p(y = +1|x)$.

Идея: ***обучаем логистическую регрессию на ответах классификатора*** $\boldsymbol{a(x)}$.

- $\pi(x; \alpha; \beta) = \sigma(\alpha \cdot \boldsymbol{a(x)} + \beta) = \frac{1}{1+e^{-(\alpha \cdot a(x)+\beta)}}$

КАЛИБРОВКА ПЛАТТА

- Пусть есть два класса, $Y = \{+1, -1\}$

Задача: для классификатора $a(x)$, предсказывающего значения из отрезка $[0, 1]$, либо предсказывающего класс (+1 или -1), сделать калибровку, чтобы предсказания были вероятностями $p(y = +1|x)$.

Идея: ***обучаем логистическую регрессию на ответах классификатора*** $a(x)$.

- $\pi(x; \alpha; \beta) = \sigma(\alpha \cdot a(x) + \beta) = \frac{1}{1+e^{-(\alpha \cdot a(x)+\beta)}}$
- Находим α и β, минимизируя логистическую функцию потерь (*то есть обучаем логистическую регрессию*):

$$-\sum_{y_i=-1} \log\big(1 - \pi(x; \alpha; \beta)\big) - \sum_{y_i=+1} \log\big(\pi(x; \alpha; \beta)\big) \to \min_{\alpha,\beta}$$

Чем плох случайный лес?

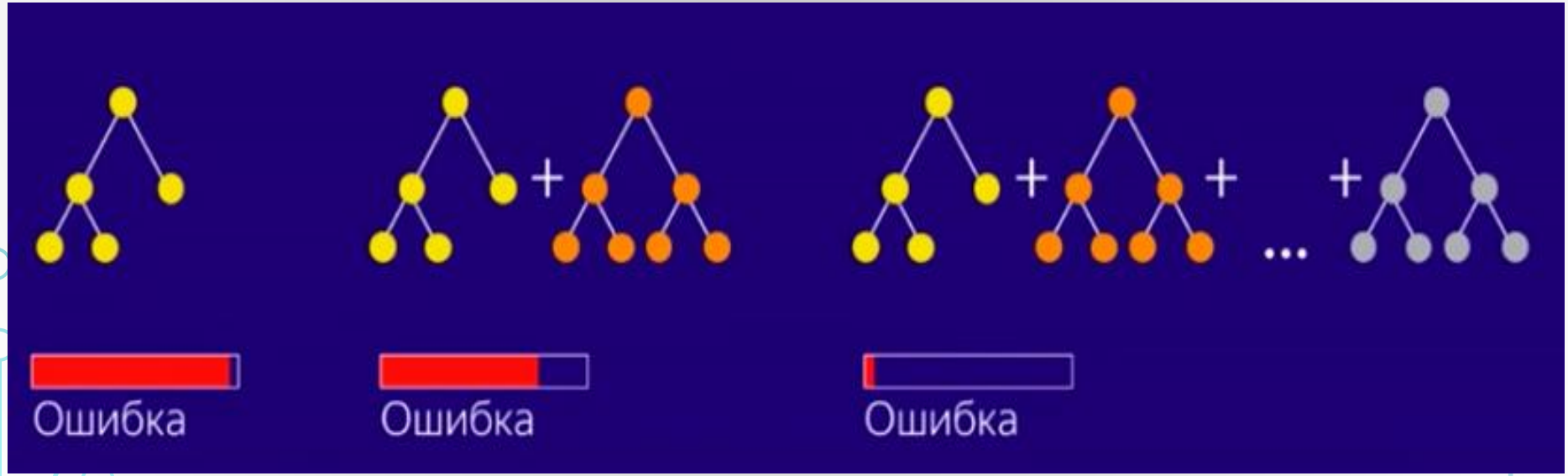
- Нужны глубокие деревья, могут очень долго обучаться
- Если одно дерево не справляется с задачей, то усреднение вряд ли поможет

БУСТИНГ

Идея: строим набор алгоритмов, каждый из которых исправляет ошибку предыдущих.

БУСТИНГ

Идея: строим набор алгоритмов, каждый из которых исправляет ошибку предыдущих.



БУСТИНГ В ЗАДАЧЕ РЕГРЕССИИ

Решаем задачу регрессии с минимизацией квадратичной ошибки:

$$\frac{1}{2}\sum_{i=1}^{l}(a(x_i)-y_i)^2 \to \min_a$$

Ищем алгоритм $a(x)$ в виде суммы N базовых алгоритмов:

$$a(x)=\sum_{n=1}^{N} b_n(x)\,,$$

где базовые алгоритмы $b_n(x)$ принадлежат некоторому семейству A.

БУСТИНГ В ЗАДАЧЕ РЕГРЕССИИ

Шаг 1: Ищем алгоритм $b_1(x)$, минимизирующий ошибку:

$$b_1(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - \boldsymbol{y_i})^2$$

- Ошибка на объекте x:

$$s = y - b_1(x)$$

БУСТИНГ В ЗАДАЧЕ РЕГРЕССИИ

Шаг 1: Ищем алгоритм $b_1(x)$, минимизирующий ошибку:

$$b_1(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - y_i)^2$$

- Ошибка на объекте x:

$$\boldsymbol{s} = y - b_1(x)$$

Следующий алгоритм должен настраиваться на эту ошибку, т.е. ***целевая переменная для следующего алгоритма – это вектор ошибок*** $\boldsymbol{s}$ (а не исходный вектор y)

БУСТИНГ В ЗАДАЧЕ РЕГРЕССИИ

Шаг 1: Ищем алгоритм $b_1(x)$, минимизирующий ошибку:

$$b_1(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - y_i)^2$$

Шаг 2: Ищем алгоритм $b_2(x)$, настраивающийся на ошибки s первого алгоритма:

$$b_2(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - \boldsymbol{s_i})^2$$

БУСТИНГ В ЗАДАЧЕ РЕГРЕССИИ

Шаг 1: Ищем алгоритм $b_1(x)$, минимизирующий ошибку:

$$b_1(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - y_i)^2$$

Шаг 2: Ищем алгоритм $b_2(x)$, настраивающийся на ошибки s первого алгоритма:

$$b_2(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} (b(x_i) - s_i)^2$$

Следующий алгоритм $b_3(x)$ будем выбирать так, чтобы он минимизировал ошибку предыдущей композиции (т.е. $b_1(x) + b_2(x)$) и т.д.

БУСТИНГ В ЗАДАЧЕ РЕГРЕССИИ

Каждый следующий алгоритм настраиваем на ошибку предыдущих.

Шаг N: Ошибка: $\boldsymbol{s_i^{(N)}} = y_i - \sum_{n=1}^{N-1} b_n(x_i) = y_i - a_{N-1}(x_i)$

Ищем алгоритм $b_N(x)$:

$$b_N(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} \left(b(x_i) - \boldsymbol{s_i^{(N)}} \right)^2$$

БУСТИНГ В ЗАДАЧЕ РЕГРЕССИИ

Каждый следующий алгоритм настраиваем на ошибку предыдущих.

Шаг N: Ошибка: $s_i^{(N)} = y_i - \sum_{n=1}^{N-1} b_n(x_i) = y_i - a_{N-1}(x_i)$

Ищем алгоритм $b_N(x)$:

$$b_N(x) = \underset{b \in A}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{l} \left(b(x_i) - s_i^{(N)} \right)^2$$

Утверждение. Ошибка на N-м шаге – это антиградиент функции потерь по ответу модели, вычисленный в точке ответа уже построенной композиции:

$$s_i^{(N)} = y_i - a_{N-1}(x_i) = -\frac{\partial}{\partial z} \frac{1}{2} (z - y_i)^2 \Big|_{z = a_{N-1}(x_i)}$$

ГРАДИЕНТНЫЙ БУСТИНГ

Пусть $L(y, z)$ – произвольная дифференцируемая функция потерь. Строим алгоритм $a_N(x)$ вида

$$\boldsymbol{a_L(x) = \sum_{n=1}^{L} \gamma_n b_n(x)}$$

ГРАДИЕНТНЫЙ БУСТИНГ

Пусть $L(y, z)$ – произвольная дифференцируемая функция потерь. Строим алгоритм $a_N(x)$ вида

$$a_L(x) = \sum_{n=1}^{L} \gamma_n b_n(x),$$

где на N-м шаге

$$b_N(x) = \underset{b \in A}{\operatorname{argmin}} \sum_{i=1}^{l} \left(b(x_i) - s_i^{(N)} \right)^2,$$

$$s_i^{(N)} = y_i - a_{N-1}(x_i)?$$

ГРАДИЕНТНЫЙ БУСТИНГ

Пусть $L(y, z)$ – произвольная дифференцируемая функция потерь. Строим алгоритм $a_N(x)$ вида

$$a_L(x) = \sum_{n=1}^{L} \gamma_n b_n(x),$$

где на N-м шаге

$$b_N(x) = \underset{b \in A}{\operatorname{argmin}} \sum_{i=1}^{l} \left(b(x_i) - s_i^{(N)} \right)^2,$$

~~$s_i^{(N)} = y_i - a_{N-1}(x_i)$~~ $\quad s_i^{(N)} = -\dfrac{\partial L}{\partial z}$

ГРАДИЕНТНЫЙ БУСТИНГ

Пусть $L(y, z)$ – произвольная дифференцируемая функция потерь.

Строим алгоритм $a_N(x)$ вида

$$a_L(x) = \sum_{n=1}^{L} \gamma_n b_n(x),$$

где на N-м шаге

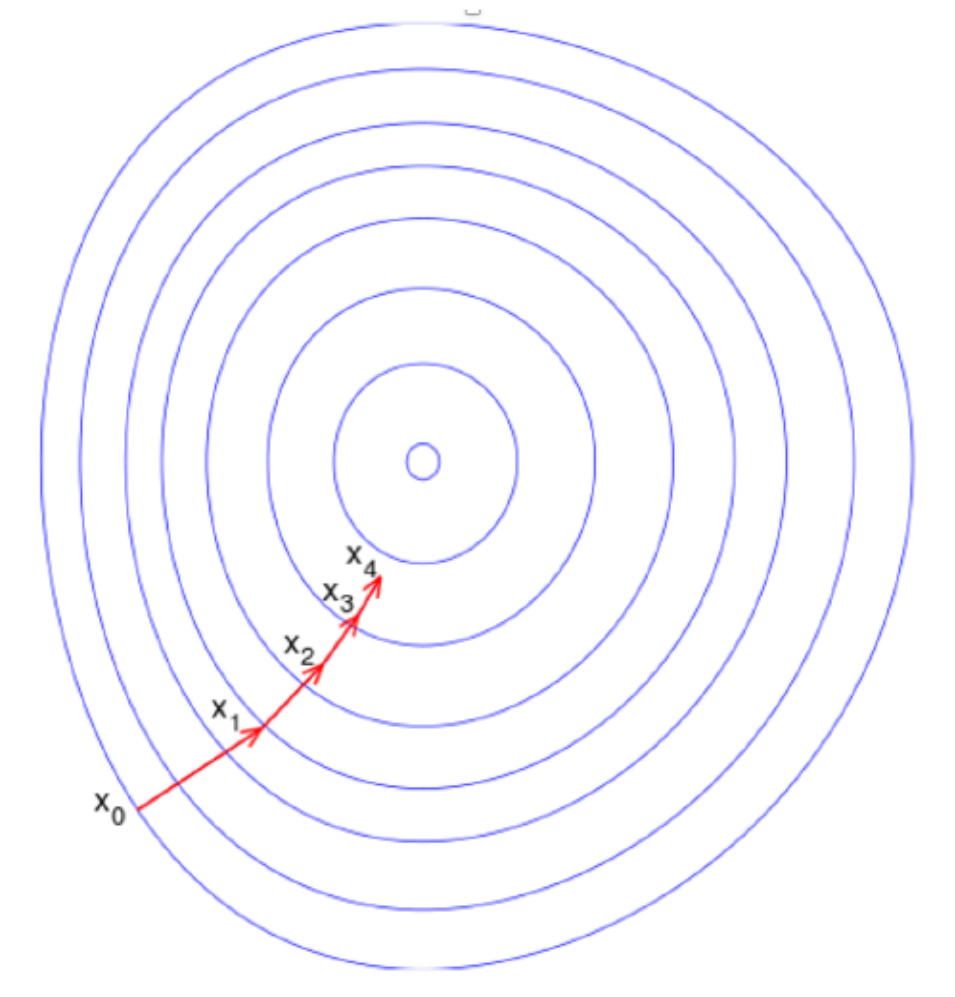
$$b_N(x) = \underset{b \in A}{\operatorname{argmin}} \sum_{i=1}^{l} \left(b(x_i) - s_i^{(N)} \right)^2,$$

$$s_i^{(N)} = -\frac{\partial L}{\partial z}$$

Коэффициент γ_N должен минимизировать ошибку:

$$\gamma_N = \underset{\gamma \in \mathbb{R}}{\operatorname{argmin}} \sum_{i=1}^{l} L(y_i, a_{N-1}(x_i) + \gamma_N b_N(x_i))$$

ГРАДИЕНТНЫЙ СПУСК В ПРОСТРАНСТВЕ ФУНКЦИЙ



ВЫБОР БАЗОВЫХ АЛГОРИТМОВ

- *Что произойдет с предсказанием бустинга, если базовые алгоритмы слишком простые?*
- *Что будет, если базовые алгоритмы слишком сложные?*

БУСТИНГ: ВЫБОР БАЗОВЫХ АЛГОРИТМОВ

- Если базовые алгоритмы очень простые, то они плохо приближают антиградиент функции потерь, т.е. градиентный бустинг может свестись к случайному блужданию.
- Если базовые алгоритмы сложные, то за несколько шагов бустинг подгонится под обучающую выборку, и получим переобученный алгоритм.

БУСТИНГ: ВЫБОР БАЗОВЫХ АЛГОРИТМОВ

- Если базовые алгоритмы очень простые, то они плохо приближают антиградиент функции потерь, т.е. градиентный бустинг может свестись к случайному блужданию.
- Если базовые алгоритмы сложные, то за несколько шагов бустинг подгонится под обучающую выборку, и получим переобученный алгоритм.

Чаще всего в качестве базовых алгоритмов используют ***решающие деревья***.

БУСТИНГ: ВЫБОР БАЗОВЫХ АЛГОРИТМОВ

- Если базовые алгоритмы очень простые, то они плохо приближают антиградиент функции потерь, т.е. градиентный бустинг может свестись к случайному блужданию.
- Если базовые алгоритмы сложные, то за несколько шагов бустинг подгонится под обучающую выборку, и получим переобученный алгоритм.

Чаще всего в качестве базовых алгоритмов используют ***решающие деревья***.

В таком случае *решающие деревья не должны быть очень маленькими, а также очень глубокими*.

Оптимальная глубина – от 3 до 6 (зависит от задачи).

СОКРАЩЕНИЕ ШАГА (РЕГУЛЯРИЗАЦИЯ)

- Если базовые алгоритмы очень простые, то они плохо приближают антиградиент функции потерь, т.е. градиентный бустинг может свестись к случайному блужданию.
- Если базовые алгоритмы сложные, то за несколько шагов бустинг подгонится под обучающую выборку, и получим переобученный алгоритм.

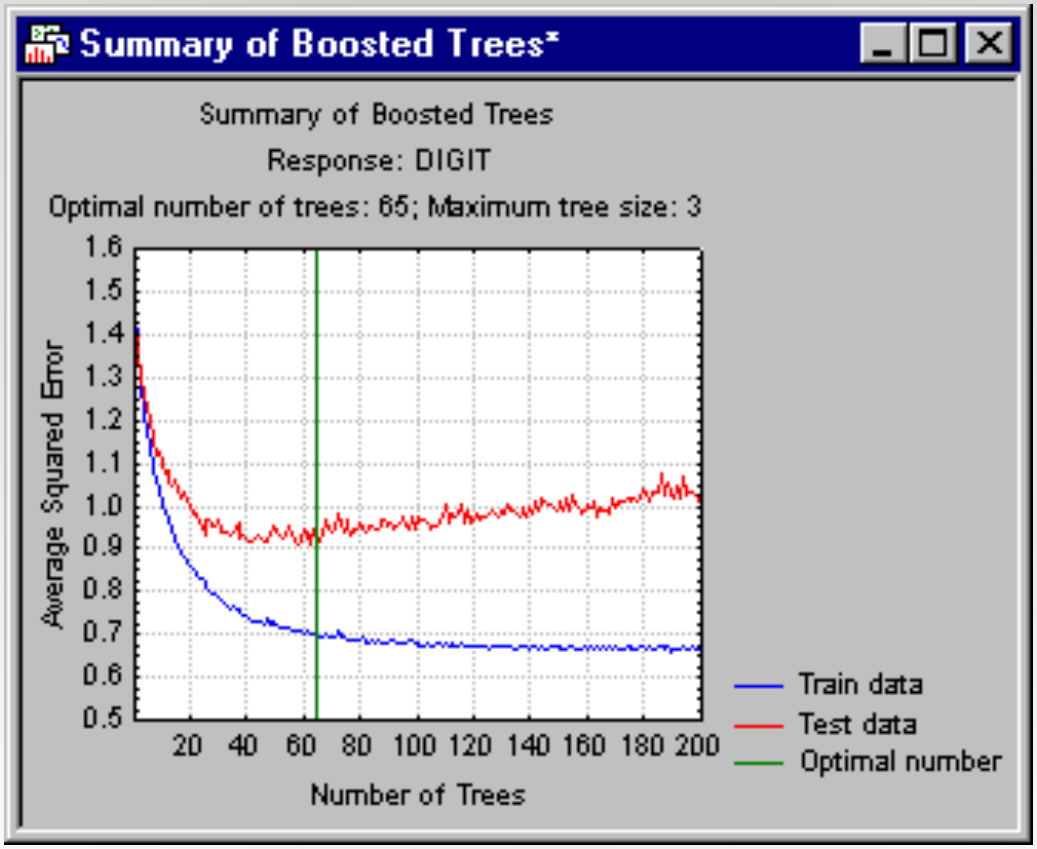
Возможное решение – сокращение шага:

$$a_N(x) = a_{N-1}(x) + \boldsymbol{\eta}\gamma_N b_N(x), \eta \in (0; 1]$$

Чем меньше темп обучения η, тем меньше степень доверия к каждому базовому алгоритму, и тем лучше качество итоговой композиции.

КОЛИЧЕСТВО ИТЕРАЦИЙ БУСТИНГА

Так как на каждом шаге бустинга целенаправленно уменьшается ошибка на тренировочной выборке, то если процесс не остановить, то мы достигнем нулевой ошибки, а значит, переобучимся!



СТОХАСТИЧЕСКИЙ ГРАДИЕНТНЫЙ БУСТИНГ

- Будем обучать базовый алгоритм b_N не по всей выборке X, а по случайной подвыборке $X^k \subset X$.

+: снижается уровень шума в данных

+: вычисления становятся быстрее

Обычно берут $\left|X^k\right| = \frac{1}{2}|X|$.

СМЕЩЕНИЕ И РАЗБРОС БУСТИНГА

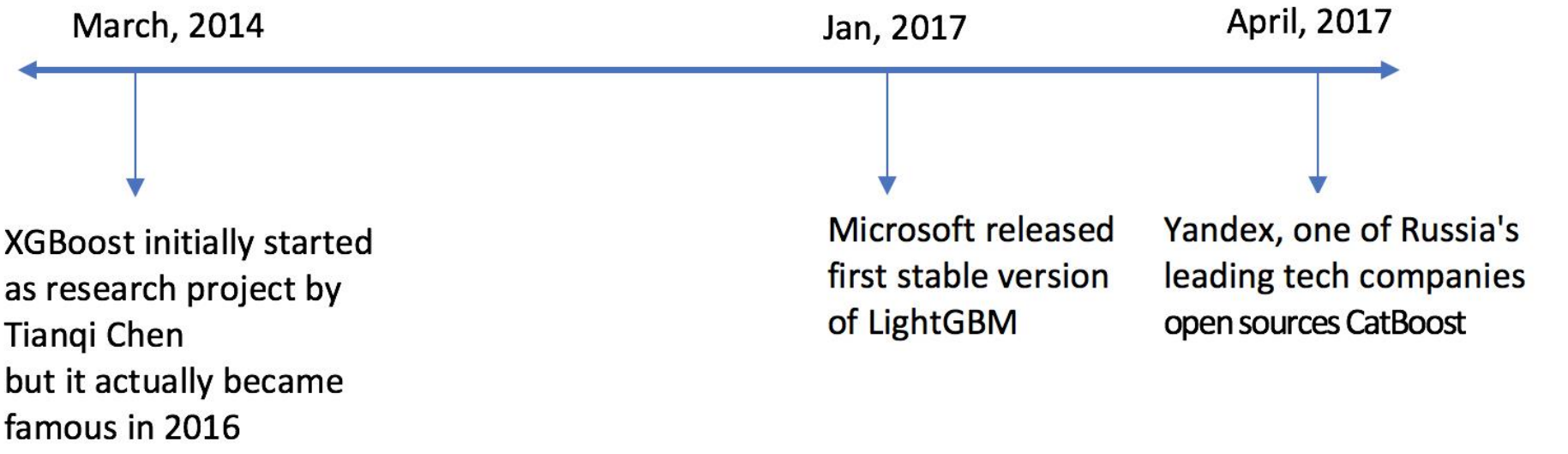
- Бустинг целенаправленно уменьшает ошибку, т.е. смещение у него маленькое.
- Алгоритм получается сложным, поэтому разброс может быть большим.

Значит, чтобы не переобучиться, в качестве базовых алгоритмов надо брать неглубокие деревья (глубины 3-6).

ИМПЛЕМЕНТАЦИИ ГРАДИЕНТНОГО БУСТИНГА

- Xgboost
- CatBoost
- LightGBM

XGBOOST, LIGHTGBM, CATBOOST



- https://github.com/dmlc/xgboost
- https://github.com/Microsoft/LightGBM
- https://towardsdatascience.com/catboost-vs-light-gbm-vs-xgboost-5f93620723db

XGBOOST (EXTREME GRADIENT BOOSTING)

- На каждом шаге градиентного бустинга решается задача

$$\sum_{i=1}^{l}(b(x_i)-s_i)^2 \to \min_b$$

$$\Leftrightarrow \sum_{i=1}^{l}\left(-s_i b(x_i)+\frac{1}{2}b^2(x_i)\right)^2 \to \min_b$$

- На каждом шаге xgboost решается задача

$$\sum_{i=1}^{l}\left(-s_i b(x_i)+\frac{1}{2}h_i b^2(x_i)\right)+\gamma J+\frac{\lambda}{2}\sum_{j=1}^{J}b_j^2 \to \min_b, \quad (*)$$

$$h_i=\frac{\partial^2 L}{\partial z^2}\Big|_{a_{N-1}(x_i)}$$

XGBOOST

$$\sum_{i=1}^{l}\left(-s_i b(x_i)+\frac{1}{2}\boldsymbol{h_i} b^2(x_i)\right)+\gamma J+\frac{\lambda}{2}\sum_{j=1}^{J} b_j^2 \to \min_b$$

Основные особенности xgboost:

- базовый алгоритм приближает направление, посчитанное с учетом *второй производной* функции потерь
- функционал *регуляризуется* – добавляются штрафы за количество листьев и за норму коэффициентов
- при построении дерева используется критерий информативности, зависящий от оптимального вектора сдвига
- критерий останова при обучении дерева также зависит от оптимального сдвига

CATBOOST

CatBoost – алгоритм, разработанный в Яндексе. Он является оптимизацией Xgboost и в отличие от Xgboost умеет обрабатывать категориальные признаки.

https://github.com/catboost/catboost

CATBOOST

Особенности catboost:

- используются симметричные деревья решений



CATBOOST

Особенности catboost:

- Для кодирования категориальных признаков используется набор методов (one-hot encoding, счётчики, комбинации признаков и др.)

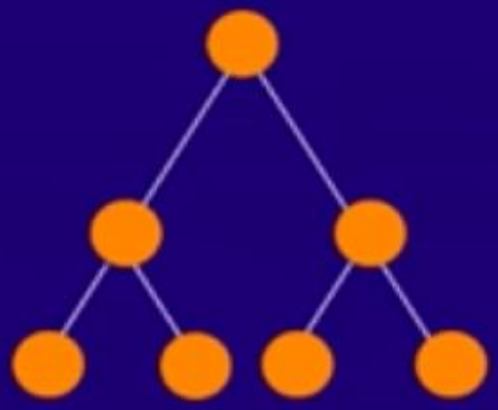


CATBOOST

Особенности catboost:

- динамический бустинг

Динамический бустинг



$$\text{leafValue(doc)} = \sum_{i=1}^{doc} \frac{g(\text{approx(i)}, \text{target(i)})}{docs\ in\ the\ past}$$

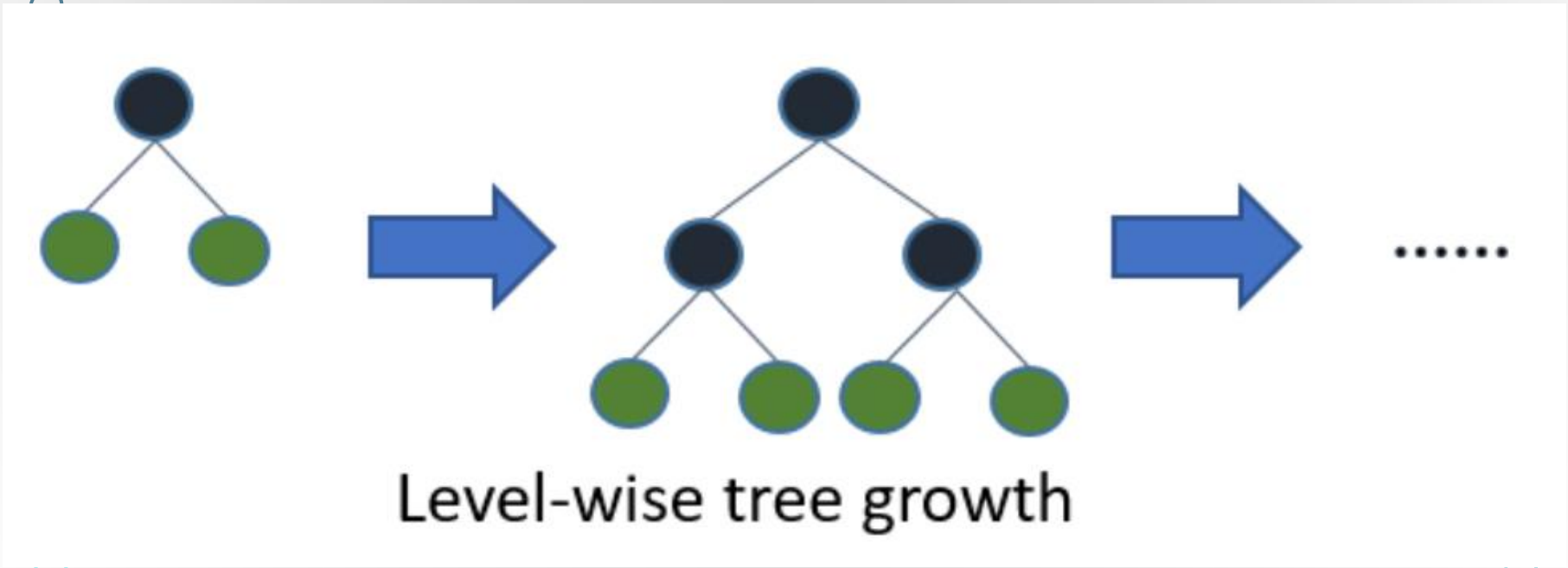
CATBOOST

Бонусы реализации:

- Поддержка пропусков в данных
- Обучается быстрее, чем xgboost
- Показывает хороший результат даже без подбора параметров
- Удобные методы: проверка на переобученность, вычисление значений метрик, удобная кросс-валидация и др.

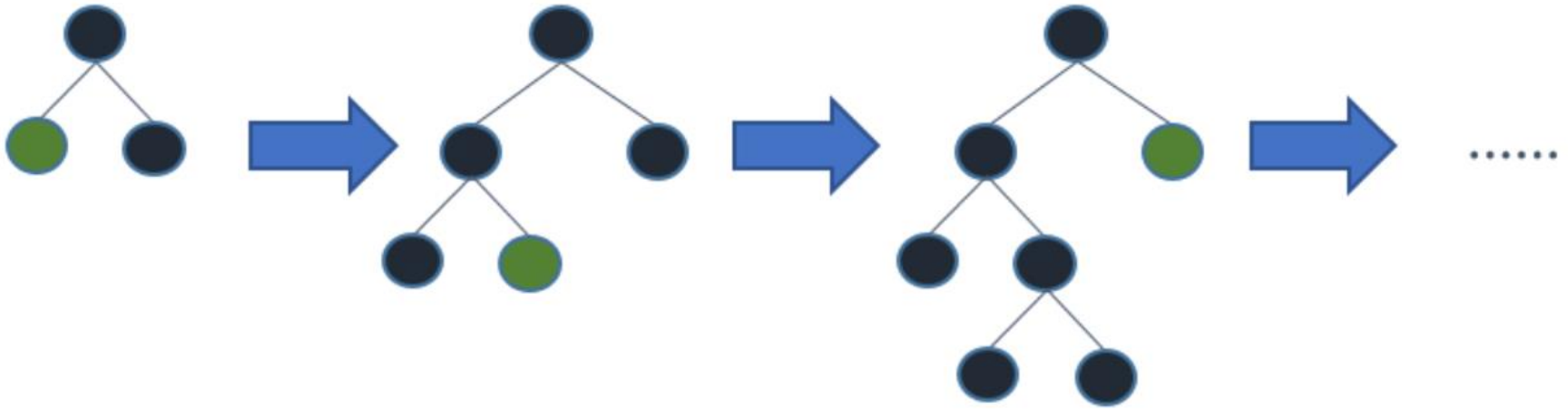
LIGHTGBM

В других реализациях градиентного бустинга деревья строятся по уровням:



LIGHTGBM

LightGBM строит деревья, добавляя на каждом шаге один лист:



Leaf-wise tree growth

Такой подход позволяет добиться более высокой точности решения задачи оптимизации.

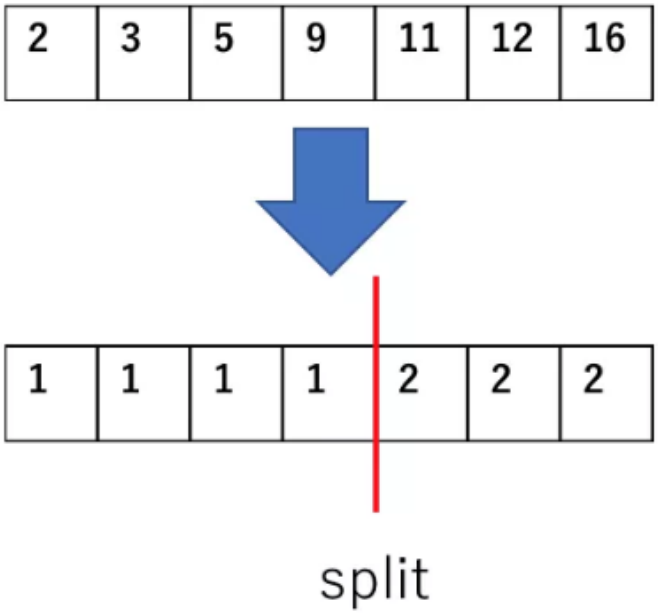
LIGHTGBM

Кодирование категориальных признаков.

- LightGBM разбивает значения категориального признака на два подмножества в каждой вершине дерева, находя при этом наилучшее разбиение
- Если категориальный признак имеет k различных значений, то возможных разбиений $2^{k-1} - 1$. В LightGBM реализован способ поиска оптимального разбиения за $O(klogk)$ операций.

LIGHTGBM

Ускорение построения деревьев за счёт бинаризации признаков:



An example of how binning can reduce the number of splits to explore. The features must be sorted in advance for this method to be effective.

Бинаризация числовых признаков:
Например, если у вас есть признак "возраст", вы можете бинаризовать его, установив порог, например, 18 лет:
Если возраст >= 18, то значение = 1 (взрослый).
Если возраст < 18, то значение = 0 (несовершеннолетний).

Бинаризация категориальных признаков:
Для категориального признака "цвет" с возможными значениями "красный", "зеленый" и "синий" можно создать бинарные признаки:
is_red: 1, если цвет красный, иначе 0.
is_green: 1, если цвет зеленый, иначе 0.
is_blue: 1, если цвет синий, иначе 0.